ANÁLISIS DEL RETO

Juan Andrés Ospina Ramírez - J.ospinar2@uniandes.edu.co - 202614830

José Alexander Santos Cobos - j.santosc2@uniandes.edu.co - 202613656

Juan Esteban Lesmes Chaves - je.lesmesc1@uniandes.edu.co - 202614029

Requerimiento 1

Al cargar, agrupé los pedidos por producto y país en otra tabla de hash. Al consultar, fui directo a su lista, calculé los promedios y ordené por monto para mostrar los dos más altos.

Entrada	Catalog , anio, mes, descuento_min, descuento_max
Salidas	Lista “resultado” con tiempo de ejecución del requerimiento en milisegundos, número total de pedidos que cumplieron el filtro, precio promedio de Price_per_Box, promedio de Discount_Pct, promedio de Marketing_Spend, promedio de Boxes_Shipped y los dos criterios con mayor amount de los que cumplen el criterio con los datos Order_ID, Canal (Channel), Fecha (Order_Date), Cajas enviadas (Boxes_Shipped) y Monto (Amount)
Implementado (Sí/No)	Si, Jose Alexander Santos Cobos

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	**Complejidad**
Paso 1: Se inicia el cronómetro, se convierte el mes a entero y se arma la llave "YYYY-MM" concatenando textos de tamaño fijo	O(1)
Paso 2: Se busca la llave en la tabla de hash de Linear Probing y se obtiene la lista de pedidos de ese mes	O(1)
Paso 3: Se crea la lista vacía de filtrados y se inicializa el acumulador del monto	O(1)
Paso 4: Se recorren los m pedidos del mes, se lee el descuento, se compara con el rango y, si cumple, se agrega al final de filtrados y se suma su Amount	O(m), donde m = pedidos filtrados del mes
Paso 5: Se arma el diccionario de resultado con el total y el monto acumulado	O(1)
Paso 6: Se recorren los pedidos filtrados para sumar Price_per_Box y Boxes_Shipped	O(n), donde n = pedidos filtrados para Price_per_Box y Boxes_Shipped
Paso 7: Se calculan los dos promedios con una división cada uno	O(1)

Paso 8: Se ordenan los filtrados con Merge Sort con el criterio Discount_Pct descendente, si son iguales Amount descendente, y si siguen siendo iguales luego Order_ID ascendente	O(n log n), donde n = pedidos filtrados para Price_per_Box y Boxes_Shipped
Paso 9: Si hay más de 20, se toman los primeros 10 y los últimos 10 con sub_list.	O(1)
Paso 10: Se detiene el cronómetro y se calcula el tiempo	O(1)
TOTAL	***O(m + n log n)***

Análisis

Se ejecutó entregando los resultados que coincidían con el filtro, la complejidad de m + n log n resultó ser buena, demostrando un corto tiempo de ejecución y al cambiar los parámetros los datos seguían funcionando

Requerimiento 2

Plantilla para el documentar y analizar cada uno de los requerimientos.

Descripción

Breve descripción de como abordaron la implementación del requerimiento

Entrada	Catalog, país, min_cajas, max_cajas
Salidas	Un diccionario con el tiempo de ejecución, número total de pedidos que pasaron el filtro, número de productos diferentes presentes en el resultado, promedios de marketing_spend y amount, producto más frecuente, entre los pedidos filtrados y número de pedidos de este producto, para cada pedido que cumple la consulta, mostrar ordenado ascendentemente por Boxes_Shipped con lo siguiente: Order_ID, Product, Channel, Order_date, Boxes_Shipped, Price_per_box, Marketing_Spend y el amount.
Implementado (Sí/No)	Implementado por Juan Lesmes

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	**Complejidad**
Acceso al país en la tabla hash.	O(1)
Filtrar por rango de cajas y acumular	O(a)
Sacar producto más frecuente y promedios	O(b)
Ordenar con merge_sort()	O(m log m)
Armar el resultado final (primeros 10 y último 10)	O(1)
TOTAL	***O(a + m log m)***

Análisis

Análisis de resultados de la implementación, tener cuenta las pruebas realizadas y el análisis de complejidad.

El Req2 tiene una complejidad de O(a + m log m), donde a es el número de pedidos del país buscado y m los pedidos que cumplen el rango de cajas. El acceso al país en la tabla hash es O(1) promedio, el filtrado y la acumulación recorren solo los **a** pedidos de ese país (nunca la lista principal, usamos la filtrada), y el merge sort ordena los m pedidos filtrados con complejidad O(m log m). En las pruebas realizadas con el archivo del 100%, la implementación con Separate Chaining resultó aproximadamente 5 ms más rápida que la de Linear Probing. Esto se debe a que por cada pedido filtrado el requerimiento hace un get y un put sobre el mapa auxiliar de conteo por producto, y el put sobre una llave que ya existe en Linear Probing implica más llamadas por dentro que las que hay al recorrer una SLL corta en Separate Chaining. Como ambas implementaciones tienen acceso O(1) promedio, la diferencia puede ser de implementación y no de orden de complejidad.

```python
def comparacion_req2 (e1, e2):
    if e1["Boxes_Shipped"] != e2["Boxes_Shipped"]:
        return e1["Boxes_Shipped"] < e2["Boxes_Shipped"]
    if e1["Order_Date"] != e2["Order_Date"]:
        return e1["Order_Date"] > e2["Order_Date"]
    return e1["Order_ID"] < e2["Order_ID"]

def req_2(catalog, pais, min_cajas, max_cajas):
    """
    Retorna el resultado del requerimiento 2
    """
    tiempoinicial = get_time()

    lista_pais = mlp.get(catalog["pedidos_por_pais_lp"], pais)
    filtrados = al.new_list()
    conteo_productos = mlp.new_map(10, 0.5)
    suma_marketing = 0
    suma_amount = 0

    if lista_pais is not None:
        for i in range(al.size(lista_pais)):
            producto = al.get_element(lista_pais, i)
            cajas = producto["Boxes_Shipped"]
            if cajas is not None and (min_cajas <= cajas and cajas <= max_cajas):
                al.add_last(filtrados, producto)
                suma_marketing += producto["Marketing_Spend"]
                suma_amount += producto["Amount"]
                c = mlp.get(conteo_productos, producto["Product"])
                if c is None:
                    mlp.put(conteo_productos, producto["Product"], 1)
                else:
                    mlp.put(conteo_productos, producto["Product"], c + 1)

    totalfiltrados = al.size(filtrados)
    resultado = {
        "total": totalfiltrados,
        "n_productos": 0,
        "promedio_marketing": 0,
        "promedio_amount": 0,
        "producto_top": None,
        "pedidos_top": 0,
        "lista": al.new_list()
    }
```

```
    }
    if totalfiltrados > 0:
        claves = mlp.key_set(conteo_productos)
        resultado["n_productos"] = al.size(claves)

        mejor = None
        mejor_cuenta = 0
        for i in range(al.size(claves)):
            producto2 = al.get_element(claves, i)
            cuenta = mlp.get(conteo_productos, producto2)
            if cuenta > mejor_cuenta or (cuenta == mejor_cuenta and producto2 < mejor):
                mejor = producto2
                mejor_cuenta = cuenta

        resultado["producto_top"] = mejor
        resultado["pedidos_top"] = mejor_cuenta
        resultado["promedio_marketing"] = suma_marketing / totalfiltrados
        resultado["promedio_amount"] = suma_amount / totalfiltrados

        ordenados = al.merge_sort(filtrados, comparacion_req2)
        if totalfiltrados > 20:
            salida = al.new_list()
            for i in range(10):
                al.add_last(salida, al.get_element(ordenados, i))
            for i in range(totalfiltrados - 10, totalfiltrados):
                al.add_last(salida, al.get_element(ordenados, i))
            resultado["lista"] = salida
        else:
            resultado["lista"] = ordenados

    tiempofinal = get_time()
    resultado["tiempo"] = delta_time(tiempoinicial, tiempofinal)

    return resultado
```

Requerimiento 3

```
def cmp_req3(p1, p2):
        if p1["Amount"] > p2["Amount"]:
            return True
        elif p1["Amount"] == p2["Amount"]:
            aporte1 = p1["Amount"] - p1["Marketing_Spend"]
            aporte2 = p2["Amount"] - p2["Marketing_Spend"]
            if aporte1 > aporte2:
                return True
            elif aporte1 == aporte2:
                return p1["Order_ID"] < p2["Order_ID"]
        return False
def req_3(catalog, fecha, min_amount, max_amount):
    """
    Retorna el resultado del requerimiento 3
    """
    inicio=get_time()
    filtrados = al.new_list()
    lista_pais = msc.get(catalog["pedidos_por_fecha"], fecha)


    i=0
    pedido = lt.get_element(lista_pais, i)
    while pedido is not None and i < lt.size(lista_pais):
        pedido = lt.get_element(lista_pais, i)
        if min_amount <= pedido["Amount"] <= max_amount:
            al.add_last(filtrados, pedido)

        i += 1
    if filtrados is None:
        print("No hay pedidos en esa fecha.")
        return None

    total_amount=0
    total_marketing=0
    for pedidos in filtrados["elements"]:
        total_amount+=pedidos["Amount"]
        total_marketing+=pedidos["Marketing_Spend"]

    total_aporte=total_amount-total_marketing
    sorted_list = al.merge_sort(filtrados, cmp_req3)

    if al.size(sorted_list) > 20:
        resultado = al.sub_list(sorted_list, 1, 10)
        ultimos = al.sub_list(sorted_list, al.size(sorted_list) - 9, 10)
        for p in ultimos["elements"]:
            al.add_last(resultado, p)
```

```
else:
    resultado = sorted_list
fin=get_time()
tiempo=delta_time(inicio,fin)
print(f"Tiempo de ejecución: {tiempo} ms")
print(f"Número de pedidos filtrados: {al.size(sorted_list)}")
print(f"Monto total recaudado: {total_amount}")
print(f"Total Marketing Spend: {total_marketing}")
print(f"Total Aporte Comercial: {total_aporte}\n")

for pedidos in resultado["elements"]:
    aporte = pedidos["Amount"] - pedidos["Marketing_Spend"]
    print(pedidos["Order_ID"], pedidos["Product"], pedidos["Country"], pedidos["Order_Date"],
          "Marketing:", pedidos["Marketing_Spend"], "Boxes:", pedidos["Boxes_Shipped"],
          "Amount:", pedidos["Amount"], "Aporte:", aporte)
```

Descripción

Se implementó un algoritmo que filtra los pedidos según una fecha específica y un rango de montos. Luego, ordena los resultados por monto y aporte comercial, mostrando los primeros 10 y los últimos 10 pedidos según el criterio de ordenamiento que nos proporcionan.

Entrada	catalog, fecha, min_amount, max_amount
Salidas	Lista de pedidos filtrados y ordenados. Totales de monto, marketing y aporte comercial. Tiempo de ejecución del algoritmo.
Implementado (Sí/No)	Si, implementado por Juan Ospina

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	**Complejidad**
Acceder a la lista de pedidos por fecha desde el mapa	*O(1)*
Filtrar por rango de montos	O(n)
Calcular totales (suma de campos)	O(n)
Ordenar los pedidos filtrados con merge sort	O(n log n)
TOTAL	***O(n log n)***

Análisis

Al usar un mapa de hash que definimos desde la carga de datos nos ayuda a reducir la complejidad, ya que aunque en el peor de los casos igualmente se filtran todos los pedidos seria un caso muy raro ya que en la mayoría de los casos los pedidos no tienen la misma fecha, lo que permite que el mapa de hash si le quite complejidad al algoritmo, el resto de el algoritmo si mantiene complejidad lineal y el merge sort usa una complejidad de n log n en todos los casos, lo que hace que la complejidad final sea de n log n

Requerimiento 4

Plantilla para el documentar y analizar cada uno de los requerimientos.

Entrada	catalog, producto, pais
Salidas	Número total de pedidos que cumplen el filtro, promedio de Price_per_Box, Discount_Pct, Marketing_Spend y Boxes_Shipped, los dos pedidos de mayor Amount, tiempo de ejecución del algoritmo
Implementado (Sí/No)	Si, Jose Alexander Santos Cobos

Descripción

Cuando cargué, guardé los pedidos agrupados por producto y por país en otra tabla hash. Cuando consulté, fui a su lista, calculé los promedios y los ordené por monto mostrando los dos más altos.

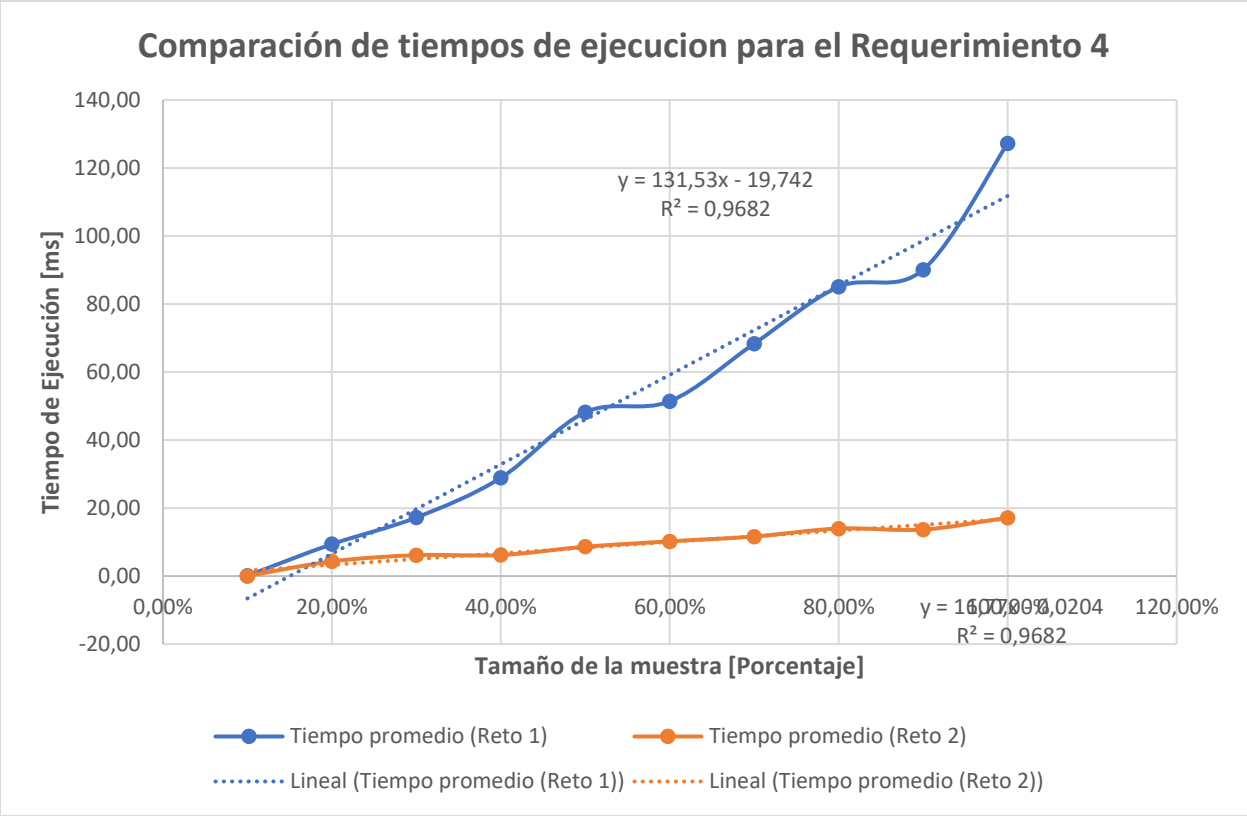
Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	**Complejidad**
Paso 1: Se inicia el cronómetro y se arma la llave "producto - pais" concatenando dos textos	O(1)
Paso 2: Se busca la llave en la tabla de hash de Linear Probing y se obtiene la lista de pedidos de esa combinación	O(1) promedio
Paso 3: Se obtiene el tamaño de la lista y se crea el diccionario de resultado con los acumuladores en cero	O(1)
Paso 4: Se recorren los p pedidos de la combinación. Por cada uno se suman, si no son None, Price_per_Box, Discount_Pct, Marketing_Spend y Boxes_Shipped	O(p)
Paso 5: Se calculan los cuatro promedios con una división cada uno	O(1)
Paso 6: Se ordenan los p pedidos con Merge Sort en orden Amount descendente, si es igual, luego Marketing_Spend ascendente, si es igual, luego Order_ID ascendente)	O(p log p)
Paso 7: Se toman los dos primeros con sub_list	O(1)
Paso 8: Se detiene el cronómetro y se calcula el tiempo	O(1)
Paso 9: Se imprimen los resultados, incluidos los dos pedidos	O(1)

TOTAL:	**O(p log p)**

Análisis



Descubrimos que con las tablas de hash se redujo considerablemente el tiempo de carga de los datos, demostrando que el método es más eficiente con hashing y más específicamente con linear probing

Requerimiento 5

Plantilla para el documentar y analizar cada uno de los requerimientos.

Descripción

Breve descripción de como abordaron la implementación del requerimiento

Entrada	Catalog, N (productos a consultar), país, fecha_inicio, fecha_fin
Salidas	Tiempo de ejecución en MS, número total de pedidos que cumplen el filtro por país y el rango de las fechas, número total de productos diferentes encontrados, la suma de amount de todos los pedidos filtrados, cada producto del top N por recaudo, mostrar el producto, el número de pedidos, el recaudo total, el Price_per_box promedio, el discount_pct y el marketing spend total.

Implementado (Sí/No)	Sí, se implementó grupalmente

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	**Complejidad**
Acceso al país en la tabla hash	O(1)
Filtrar por rango de fechas y acumular por producto	O(a)
Calcular promedios y armar la lista de acumuladores	O(n)
Ordenar con merge_sort()	O(b log b)
Tomar el top N	O(P)
TOTAL	***O(a + n log n)***

Análisis

Análisis de resultados de la implementación, tener cuenta las pruebas realizadas y el análisis de complejidad.

El Req. 5 tiene una complejidad de O(a + n log n), donde es el número de pedidos del país y n el número de productos distintos en el rango de fechas. El acceso al país es O(1) promedio, el recorrido con filtro por fechas y acumulación por producto tiene complejidad O(a) y el merge sort se aplica solo sobre los n acumuladores y no sobre los pedidos con una complejidad de O(n log n). En las pruebas con el archivo del 100%, la implementación con Linear Probing fue entre 20 y 30 ms más rápida que la de Separate Chaining. Esto es gracias a que en este requerimiento el recorrido hace casi solo operaciones get sobre el mapa de acumuladores (el put ocurre únicamente la primera vez que aparece cada producto), y con pocas llaves y factor de carga bajo pues Linear Probing tiende a encontrar el producto en la primera casilla, mientras que Separate Chaining debe recorrer los nodos de la SSL cuando hay colisiones. Aun así, ambas implementaciones son O(1) promedio por acceso, por lo que la diferencia es por cómo se haya hecho la implementación y no al orden de complejidad.

```python
def comparacion_req5 (e1, e2):
    if e1["recaudo"] != e2["recaudo"]:
        return e1["recaudo"] > e2["recaudo"]
    if e1["cajas"] != e2["cajas"]:
        return e1["cajas"] > e2["cajas"]
    return e1["producto"] < e2["producto"]

def req_5(catalog, n, pais, fecha_inicio, fecha_fin):
    """
    Retorna el resultado del requerimiento 5
    """
    tiempoinicio = get_time()

    lista_pais = msc.get(catalog["pedidos_por_pais"], pais)
    acumulados = msc.new_map(10, 0.5)
    total_pedidos = 0
    recaudo_total = 0

    if lista_pais is not None and fecha_inicio <= fecha_fin:
        for i in range(al.size(lista_pais)):
            producto = al.get_element(lista_pais, i)
            if fecha_inicio <= producto["Order_Date"] and producto["Order_Date"] <= fecha_fin:
                total_pedidos += 1
                recaudo_total += producto["Amount"]
                acum = msc.get(acumulados, producto["Product"])
                if acum is None:
                    acum = {
                        "producto": producto["Product"],
                        "pedidos": 0,
                        "cajas": 0,
                        "recaudo": 0,
                        "suma_precio": 0,
                        "suma_descuento": 0,
                        "marketing": 0
                    }
                    msc.put(acumulados, producto["Product"], acum)
                acum["pedidos"] += 1
                acum["cajas"] += producto["Boxes_Shipped"]
                acum["recaudo"] += producto["Amount"]
                acum["suma_precio"] += producto["Price_per_Box"]
                acum["suma_descuento"] += producto["Discount_Pct"]
                acum["marketing"] += producto["Marketing_Spend"]

    claves = msc.key_set(acumulados)
    n_productos = al.size(claves)
```

```
    claves = msc.key_set(acumulados)
    n_productos = al.size(claves)

    lista_acum = al.new_list()
    for i in range(n_productos):
        acum = msc.get(acumulados, al.get_element(claves, i))
        acum["promedio_precio"] = acum["suma_precio"] / acum["pedidos"]
        acum["promedio_descuento"] = acum["suma_descuento"] / acum["pedidos"]
        al.add_last(lista_acum, acum)

    ordenados = al.merge_sort(lista_acum, comparacion_req5)
    top = al.new_list()
    for i in range(min(n, n_productos)):
        al.add_last(top, al.get_element(ordenados, i))

    tiempofinal = get_time()
    print(f"tiempo: {delta_time(tiempoinicio, tiempofinal)}")
    print(f"Número total de pedidos: {total_pedidos}")
    print(f"N_productos: {n_productos}")
    print(f"Recaudo total: {recaudo_total}")

    print(f"Top {n} países con mayor eficiencia de mercadeo:")
    for p in top["elements"]:
        print(p)
```

Requerimiento 6

```
def cmp_req_6(p1, p2):
        if p1["Eficiencia"] > p2["Eficiencia"]:
            return True
        elif p1["Eficiencia"] == p2["Eficiencia"]:
            if p1["Amount_total"] > p2["Amount_total"]:
                return True
            elif p1["Amount_total"] == p2["Amount_total"]:
                return p1["Country"] < p2["Country"]
        return False

def req_6(catalog, producto, fecha_inicial, fecha_final, n):
    """
    Retorna el resultado del requerimiento 6
    """

    inicio = get_time()

    filtrados = al.new_list()
    lista_productos = msc.get(catalog["productos"], producto)

    for pedido in lista_productos["elements"]:
        if fecha_inicial <= pedido["Order_Date"] <= fecha_final:
            al.add_last(filtrados, pedido)


    paises_map = msc.new_map(100,4)
    for pedido in filtrados["elements"]:
        pais = pedido["Country"]
        pais_info = msc.get(paises_map, pais)
        if pais_info is None:
            pais_info = {
                "Country": pais,
                "Pedidos": 0,
                "Amount_total": 0,
                "Marketing_total": 0,
                "Boxes_total": 0
            }
            msc.put(paises_map, pais, pais_info)
        pais_info["Pedidos"] += 1
```

```
    pais_info["Pedidos"] += 1
    pais_info["Amount_total"] += pedido["Amount"]
    pais_info["Marketing_total"] += pedido["Marketing_Spend"]
    pais_info["Boxes_total"] += pedido["Boxes_Shipped"]

paises_lista = al.new_list()
for bucket in paises_map["table"]:
    nodo = bucket["first"]
    while nodo is not None:
        entry = nodo["info"]
        info = entry["value"]
        if info["Marketing_total"] > 0:
            info["Eficiencia"] = info["Amount_total"] / info["Marketing_total"]
            al.add_last(paises_lista, info)
        nodo = nodo["next"]

paises_ordenados = al.merge_sort(paises_lista, cmp_req_6)
n_total = al.size(paises_ordenados)
if n > n_total:
    n = n_total
top_n = al.sub_list(paises_ordenados,0, n)

total_amount = sum(p["Amount_total"] for p in paises_lista["elements"])
total_marketing = sum(p["Marketing_total"] for p in paises_lista["elements"])

fin = get_time()
tiempo = delta_time(inicio,fin)

print(f"Tiempo de ejecución: {tiempo} ms")
print(f"Número total de pedidos filtrados: {al.size(filtrados)}")
print(f"Número total de países encontrados: {n_total}")
print(f"Amount total: {total_amount}")
print(f"Marketing total: {total_marketing}\n")
```

```
  print(f"Top {n} países con mayor eficiencia de mercadeo:")
  for p in top_n["elements"]:
      print(p["Country"],
            "Pedidos:", p["Pedidos"],
            "Amount:", p["Amount_total"],
            "Marketing:", p["Marketing_total"],
            "Boxes:", p["Boxes_total"],
            "Eficiencia:", round(p["Eficiencia"], 2))
```

Descripción

Se desarrolló un algoritmo que filtra los pedidos por producto y rango de fechas, agrupa los resultados por país y calcula total de pedidos, monto, marketing, cajas y eficiencia. Finalmente, ordena los países por eficiencia de mercadeo y muestra los más destacados

Entrada	producto: Nombre del producto a analizar. Fecha inicial: Fecha mínima del rango. Fecha final: Fecha máxima del rango. n: Número de países destacados a mostrar. catalogo: Estructura principal con los pedidos cargados.
Salidas	Lista de países con sus métricas requeridas. Top n países con mayor eficiencia de mercadeo. Totales globales de monto y marketing. Tiempo de ejecución del algoritmo.
Implementado (Sí/No)	Si, se implemento

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	**Complejidad**
Usar el mapa por productos para filtrar	O(1)
Filtrar por rango de fechas	O(n)
Agrupar por país y acumular métricas	O(n)
Calcular eficiencia y ordenar países	O(m log m), donde m = número de países
TOTAL	***O(n + m log m)***

Análisis

En el algoritmo usamos separate chaining para organizar los pedidos por productos, lo que nos ayuda a mantener el filtro de por productos en una complejidad O(1), lo que ya reduce en buena parte la complejidad, el otro filtro se mantiene en O(N) y al final el organizar por países la complejidad pasa a ser la de el algoritmo de ordenamiento, lo tomamos como un numero diferente porque consideramos muy raro que para cada producto que se encuentre sea de un país diferente, por eso la complejidad final seria O(N+ M LOG M)